CSE233 Week 5: Introduction to Functions

This week, we will introduce functions. Functions are a mechanism to modularize and reuse code. We'll talk about using predefined functions, as well as defining our own functions.

Functions

A function is a block of code that executes a task. Functions have their own scope, meaning that variables defined inside a function are local to that function's body. To pass data, functions accept parameters as input and return values that can be used elsewhere in the program. A function's signature consists of its return type, name, and parameter list. In C++, functions can be declared and implemented separately. Below is the general syntax for a function declaration:

```
return_type function_name(param_type1 param_name1, param_type2 param_n
```

First, the return type of the function is provided. If the function does not return a value, the `void` type is used. Then, the name of the function is provided. Finally, the parameter list. The parameter list is a comma-separated list wrapped in parentheses. If the function has no parameters, the parentheses are empty. Each parameter has a data type and a name. The function declaration ends with a semicolon. The point of a function declaration is to act as a promise that the function exists somewhere. It provides the function's signature so that the function can be called, without needing to immediately define it (the main reason why function declarations exist in C and C++ has to do with how C programs are compiled. We'll discuss the specifics of this at the end of the course).
Here is an example function declaration for a function that adds two integers, and returns an integer result:

```
int add(int a, int b);
```

In this case, the return type of the function is `int`. The name of the function is `add`. The function has two parameters, `int a` and `int b`. Here is a function declaration for a function that prints a greeting:

```
void printGreeting();
```

Since the function is only printing a greeting, and not processing data, it does not need to return a value. Therefore, the "return type" is `void`. The name of the function is `printGreeting`. This function has no parameters.
At some point, the function needs to be defined (given a body) to specify the instructions the function must carry out. A function definition has the following syntax:

```
return_type function_name(param_type1 param_name1, param_type2 param_n
    statement;
    // non-void functions must return a value
    return value;
}
```

You'll notice that this is the exact same as the syntax for the declaration, except instead of a semicolon the function now has a block. Inside the block are the statements that the function will execute when it is called. Non-void functions must have a return statement that specifies the value that is returned by the function. Void functions do not need a return statement, although one can be provided. The return statement for a void function consists of `return;` with no value after the `return` keyword. Below are the function definitions for `add()` and `printGreeting()`:

```
int add(int a, int b) {
    return a + b;
}

void printGreeting() {
    std::cout << "Hello, World!\n";
}
```

Let's view these functions in the context of a full program:

```
#include <iostream>

// function declarations before main()
int add(int a, int b);
void printGreeting();

// since the functions are declared before main(),
// they can be used in main, even though we have not
// defined them yet. if the functions were not declared here,
// they would not be usable in main()
int main() {
    // to call a void function, you put the name of the
    // function, it's argument list, and a semicolon
    // since printGreeting expects 0 parameters,
    // the argument list is empty
    printGreeting();

    // declare variables
    int x = 4;
    int y = 2;
    // variables can be passed as arguments to the function
    // the value returned by add is stored in z
    int z = add(x, y);

    std::cout << x << " + " << y << " = " << z << std::endl;

    // the function can also be called directly in the cout statement
    // we can also pass literals to the function
    std::cout << "5 + 7 = " << add(5, 7) << std::endl;

    // you can also mix and match
    std::cout << "5 + 4 = " << add(5, x) << std::endl;

    // even a function call can be an argument:
    std::cout << "(5 + 7) + (2 + 4) = "
        << add(add(5, 7), add(y, x)) << std::endl;

    return 0;
}

// we are defining the functions after main()
// since they were **declared before main(),
// they are still usable in main

int add(int a, int b) {
    return a + b;
}
```

```
void printGreeting() {
    std::cout << "Hello, World\n";
}
```

In this example, the functions are declared before the `main()` function. This allows us the use the functions in main() without needing to define them.

Argument vs. Parameter

When defining the function, the variables that go in the parentheses are called parameters. These are variables that are used as placeholders for the actual values that will be passed to the function when it is called. When calling the function, the values passed to the function are called arguments. These could be literal values, variables, expressions, or even another function call (provided the function being called returns the correct data type).

Predefined Functions

We learned that we can use `#include <iostream>` to include `std::cout` for printing data and `std::cin` for reading data. We can use `#include <string>` to include the `std::string` data type to store text data. These are called include statements, a type of C preprocessor statement. The C preprocessor is a part of the C/C++ compiler that runs before the code is compiled, and it tells the compiler to go look for this file (e.g., `iostream`, `string`) and copy-paste its contents into the file that is being compiled. The actual process is a little more sophisticated, but for our purposes this explanation is sufficient. There are many other libraries for C/C++ that contain predefined functions we can use in our code. Predefined functions are functions that are written by someone else that we can use. The compiler ships with many standard libraries that contain functions that you can use. We'll discuss a few common examples in the coming subsections.

Math (`cmath`)

The math library contains various mathematics related functions. To use it, you must `#include <cmath>`. Below is a table of common functions and their descriptions:

Function	Description
`double pow(double base, double exp)`	The pow function raises its first parameter to the power that is provided as its second parameter
`double sqrt(double x)`	Returns the square root of a number
`int floor(double x)`	Rounds a number down
`int ceil(double x)`	Rounds a number up
`int round(double x)`	Rounds a number to the nearest integer
`double sin(double x)`	Returns the sine of angle x (x in radians)
`double cos(double x)`	Returns the cosine of angle x (x in radians)
`double tan(double x)`	Returns the tangent of angle x (x in radians)
`double asin(double x)`	Returns the arc sine of angle x (x in radians)
`double acos(double x)`	Returns the arc cosine of angle x (x in radians)
`double atan(double x)`	Returns the arc tangent of angle x (x in radians)
`double log(double x)`	Returns the natural log of x (ln(x))
`double log10(double x)`	Returns the log base 10 of x (log(x))
`double log2(double x)`	Returns the log base 2 of x ($\log_2(x)$)

These are just a few of the functions provided by `cmath`.

C Standard Library (`cstdlib`)

This is the C++ wrapper for C's standard library. It provides functions for a variety of tasks, including random number generation, converting strings to integers, and exiting programs when errors occur.

Function	Description
`int rand()`	Returns a pseudo-random number between 0 and `RAND_MAX`
`void srand(unsigned seed)`	Used to seed `rand()`. `rand()` will always return the same sequence of numbers for the same seed
`void abort()`	Exits program without cleaning up. Emergency program termination option
`void exit(int exit_code)`	Exits program with clean up and returns the passed `exit_code` to the operating system
`int atoi(const char* str)`	Converts a C string to an integer
`long atol(const char* str)`	Converts a C string to a long
`float atof(const char* str)`	Converts a C string to a float
`double atod(const char* str)`	Converts a C string to a double
We'll talk about C strings in a couple of weeks.	

Time (`ctime`)

This is a C++ wrapper for the C `time` library. It contains functions for

accessing and manipulating times. C and C++ use the number of seconds from January 1st, 1970 (called the UNIX Epoch) to determine the time. It is much more efficient for a computer to process time this way than to consider the individual time parts. The `std::time_t` data type is an alias for the largest possible integer on the system used by the time library.

Function	Description
`time_t time(std::time_t* arg)`	Returns the current system time, and optionally stores the system time in `arg`
`double difftime(std::time_t time_end, std::time_t time_begin)`	Returns the difference between `time_end` and `time_start` in seconds

Often, the current time is used as a seed to the `srand()` function. This ensures that each time the program is run, a different seed is provided for pseudo-random number generation. Below is an example:

```
// include cstdlib for std::rand() and std::srand()
#include <cstdlib>
// include ctime for std::time()
#include <ctime>
#include <iostream>

int main() {
    // seed the random number generator
    // since we don't need to store the result of time,
    //  we pass it 0 as an argument and call it directly
    std::srand(std::time(0));

    // generate 10 random numbers
    for (int i = 0; i < 10; ++i) {
        std::cout << "Random number " << i + 1
            << ": " << std::rand() << std::endl;
    }

    // that generates numbers within the range 0 to RAND_MAX
    // the value of RAND_MAX varies depending on the system,
    //  but is very large nonetheless

    // if we want constrain the random number to a range of values,
    //  we can use the modulus operator %
```

```
    // this loop prints 10 random numbers from 0 to 25
    for (int i = 0; i < 10; ++i) {
        std::cout << "Random number " << i + 1
            << ": " << std::rand() % 26 << std::endl;
    }

    // std::rand() % 26 gives the remainder of std::rand() when
    //  divided by 26. the minimum remainder is 0 and the max is 25,
    //  so the values that get printed out are constrained to 0-25

    // if we want to print numbers from 1 to 25, we need to shift
    //  the output of std::rand() up by one. However, that would
    //  increase the output of 25 to 26, so we also need to change
    //  the modulus value to be one less:

    for (int i = 0; i < 10; ++i) {
        std::cout << "Random number " << i + 1
            << ": " << (std::rand() % 25) + 1 << std::endl;
    }

    // in this case, std::rand() % 25 returns numbers from 0-24
    //  whatever result we get will have 1 added to it, so the
    //  final range is shifted up by 1, and we get 1-25

    return 0;
}
```

Task Decomposition and Refactoring

Writing a program requires decomposing the task you have been given into more manageable sub-tasks. Then, you can solve the individual sub-tasks until you've created a program that solves the whole task you've been given. However, the resulting code of the program will often be messy, inefficient, and disorganized. Refactoring is the process in which you go through after writing the initial code and clean up the program. Next week, when we discuss testing and test driven development, you'll see a couple of longer examples of this. Here, I'll stick with a shorter example.

Let's say that we have been given the task of writing a program that, given a range of integers, finds the integer that takes the most steps to reach 1 following the Collatz conjecture. The Collatz conjecture states the following: Given a positive integer, if it is even, divide it by 2. If it is odd, multiply it by 3 and subtract 1. Every integer will eventually reach 1 by doing this. This has never been formally proved, which is why it is

called a conjecture and not a theorem, but there has never been an example to disprove it, either.

Let's break down the requirements of the program into manageable tasks:
1. Read two numbers 2. Apply the Collatz conjecture to each number 3. Count the number of steps to reduce the number to 1 4. Keep track of the number that took the most steps 5. Report the number that took the most steps to reach 1

With these tasks in mind, let's start implementing a solution. The easiest thing to do is to read the two numbers in, so let's do that:

```
#include <iostream>

int main() {
    unsigned start;
    unsigned end;
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";
    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    return 0;
}
```

That's task 1 done. However, in writing this, I thought of a problem. While I am asking for the minimum and maximum number explicitly, the user could still enter them in the wrong order. They could also enter the same number twice. In that case, we'd only be checking one number. This is technically valid, so we'll allow the user to enter the same number, so the program can be used to check how many steps it takes to reduce a single number to 1. However, we should validate that the start is less than or equal to the end. We could allow the user to continually reenter numbers, but I'd rather just report the invalid usage and end the program. It is short program that only asks the user to enter two numbers, so forcing them to run it again to use it correctly is easier than writing the code to prompt the user to enter the correct value.

```cpp
#include <iostream>

int main() {
    unsigned start;
    unsigned end;
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";
    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    // add logic to report invalid range
    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        return 1;
    }

    return 0;
}
```

Now, we can test if the program works. Below are some example inputs (the explanation message is excluded):

```
./collatz
Enter start of range: 1
Enter end of range: 5
./collatz
Enter start of range: 5
Enter end of range: 1
Invalid usage. Start must be less than end
./collatz
Enter start of range: 0
Enter end of range: 5
```

In testing it, I discovered another problem. 0 is not a valid input for the program. While the `unsigned` integer data type is used to prevent negative numbers from being input, 0 is also invalid, since it is not positive. 0 is neither positive nor negative. Therefore, we also need a check that neither element of the range is 0.

```cpp
#include <iostream>

int main() {
    unsigned start;
```

```
    unsigned end;
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";

    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    // add logic to ensure neither part of the range is 0
    if (start == 0 || end == 0) {
        std::cout << "Invalid usage. 0 is not a valid input value\n";
        return 1;
    }

    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        return 2;
    }

    return 0;
}
```

Now, we can test it again:

```
./collatz
Enter start of range: 0
Enter end of range: 5
Invalid usage. 0 is not a valid input value
```

Now, we've handled getting input from the user. Next, we need to apply the Collatz conjecture to each of them. First, let's loop through the numbers:

```
#include <iostream>

int main() {
    unsigned start;
    unsigned end;
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";

    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;
```

```
    if (start == 0 || end == 0) {
        std::cout << "Invalid usage. 0 is not a valid input value\n";
        return 1;
    }

    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        return 2;
    }

    // for now, let's just verify that we
    // loop through the numbers correctly
    for (unsigned i = start; i <= end; ++i) {
        std::cout << i << "\n";
    }

    return 0;
}
```

Before implementing anything else, we can check to make sure the loop works as intended:

```
./collatz
Enter start of range: 1
Enter end of range: 5
1
2
3
4
5
Enter start of range: 500
Enter end of range: 500
500
```

Once we are satisfied, we can eliminate the print statement from the loop. Now, we need to write the algorithm for the Collatz conjecture. Rather testing it on each number in the range, we will comment out the for loop for now, and just run the algorithm on the first element of the range. That way, we can test if it works before running it on each number in the range.

```
#include <iostream>

int main() {
    unsigned start;
```

```
    unsigned end;
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";

    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    if (start == 0 || end == 0) {
        std::cout << "Invalid usage. 0 is not a valid input value\n";
        return 1;
    }

    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        return 2;
    }

    // for now, let's just verify that we
    // that the algorithm we write for the Collatz
    // conjecture works as intended
    // therefore, we'll ignore looping through each
    // number in the range for now

    // for (unsigned i = start; i <= end; ++i) {
    //     std::cout << i << "\n";
    // }

    // the stop condition for the Collatz conjecture
    // is when the number becomes 1
    while (start != 1) {
        // case 1: the number is even
        if (start % 2 == 0) {
            start /= 2;
        }
        // case 2: the number is odd
        else {
            start = start * 3 + 1;
        }
    }
    std::cout << "start now equals " << start << "\n";
    return 0;
}
```

We expect that for any number we enter, that start eventually becomes 1. Now, our third requirement states that we need to count the number of steps it takes for the number to reach 1. Again, before looping

through each number in the range, let's implement the counting mechanism:

```
#include <iostream>

int main() {
    unsigned start;
    unsigned end;
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";

    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    if (start == 0 || end == 0) {
        std::cout << "Invalid usage. 0 is not a valid input value\n";
        return 1;
    }

    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        return 2;
    }

    // for (unsigned i = start; i <= end; ++i) {
    //     std::cout << i << "\n";
    // }

    // variable to count the number of steps
    unsigned steps = 0;

    while (start != 1) {
        if (start % 2 == 0) {
            start /= 2;
        }
        else {
            start = start * 3 + 1;
        }
        // increase the step count
        ++steps;
    }
    std::cout << "start now equals " << start << "\n";
    std::cout << "it took " << steps << "steps\n";
    return 0;
}
```

To verify that it is correct, we can hand calculate a couple of examples:

1

1 is 1. It takes 0 steps

5

1. 5 is odd. 5 * 3 + 1 = 16
2. 16 is even. 16 / 2 = 8
3. 8 is even. 8 / 2 = 4
4. 4 is even. 4/ 2 = 2
5. 2 is even. 2/2 = 1
6. 1 is 1. It took 5 steps to get here

10

1. 10 us even. 10 / 2 = 5
2. 5 is odd. But, we know from our previous work that it takes 5 steps to get 5 to 1. Therefore, it takes 1 + 5 = 6 steps for 10 to get to 1.

13

1. 13 is odd. 13 * 3 + 1 = 40
2. 40 is even. 40 / 2 = 20
3. 20 is even. 20 / 10 = 10
4. 10 is even. But, we know from previous work that it takes 6 steps to get 10 to 1. Therefore, it takes 3 + 6 = 9 steps for 13 to get to 1.

Once satisfied with the number of test cases, we can run the program with them. Once satisfied that it works, we can move on to the next step, running the Collatz algorithm on all numbers in the range. To do so, we will move the code into the for loop, and replace `start` with `i`. **Note: this won't work, and will create an infinite loop. I am leaving it in to demonstrate debugging and how writing a program doesn't always work on the first try**

```
#include <iostream>

int main() {
    unsigned start;
    unsigned end;
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";
```

```
    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    if (start == 0 || end == 0) {
        std::cout << "Invalid usage. 0 is not a valid input value\n";
        return 1;
    }

    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        return 2;
    }

    // move Collatz algorithm into the for loop
    for (unsigned i = start; i <= end; ++i) {
        unsigned steps = 0;
        while (i != 1) {
            if (i % 2 == 0) {
                i /= 2;
            }
            else {
                i = i * 3 + 1;
            }
            ++steps;
        }
        std::cout << "i now equals " << i << "\n";
        std::cout << "it took " << steps << " steps\n";
    }
    return 0;
}
```

Let's run this for the range 5 to 6:

```
./collatz
Enter start of range: 1
Enter end of range: 5
i now equals 1
it took 5 steps
i now equals 1
it took 1 steps
i now equals 1
it took 1 steps
i now equals 1
it took 1 steps
i now equals 1
it took 1 steps
i now equals 1
```

```
...
it took 1 steps
i now equals 1
it took 1 steps^C
```

The CTRL + C keyboard input is used to stop an infinite looping program. So, what happened? The first time the loop ran, `i` started at 5, and was reduced down to 1. It took 5 steps, as expected. But, the value of `i` is now 1. When it returns to the for loop, `i` is increased to 2 by the `++i` statement. The body of the for loop once again decreases `i` to 1, and the cycle repeats. `i` is stuck being reduced to 1, increased to 2 by the for loop, then reduced back to 1 again. Therefore, to solve this, we need to perform the Collatz algorithm on a different variable that is a copy of `i`'s original value:

```
#include <iostream>

int main() {
    unsigned start;
    unsigned end;
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";

    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    if (start == 0 || end == 0) {
        std::cout << "Invalid usage. 0 is not a valid input value\n";
        return 1;
    }

    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        return 2;
    }

    for (unsigned i = start; i <= end; ++i) {
        unsigned steps = 0;
        // copy the value of i
        unsigned value = i;
        while (value != 1) {
            if (value % 2 == 0) {
                value /= 2;
            }
```

```
            else {
                value = value * 3 + 1;
            }
            ++steps;
        }
        std::cout << "value now equals " << value << "\n";
        std::cout << "it took " << steps << " steps\n";
    }
    return 0;
}
```

The program will now behave as expected:

```
./collatz
Enter start of range: 5
Enter end of range: 6
value now equals 1
it took 5 steps
value now equals 1
it took 8 steps
```

Now, the requirements did not say to print out the steps for each number in the range. We are only supposed to print out the number of steps and the number which took the most steps. Therefore, we'll eliminate the print statements in the for loop that we used for testing, now that we are confident in the logic of the program. To store the number that took the most steps, and the number of steps, we'll need two more variables. Since these need to be used outside the while loop to report the final results at the end of the program, we need to declare them before the while loop.
The minimum possible number of steps is 0, which happens if the value is 1. Therefore, we assume that the maximum number of steps taken is 0 to start off with. Then, each time we run the Collatz algorithm on a number, we check if the number of steps taken was more than the current maximum. If it is, we update the maximum number of steps and the value that caused the maximum number of steps. This will allow us to save the max steps.

```
#include <iostream>

int main() {
    unsigned start;
    unsigned end;
```

```
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";

    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    if (start == 0 || end == 0) {
        std::cout << "Invalid usage. 0 is not a valid input value\n";
        return 1;
    }

    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        return 2;
    }

    // variables to store the current maximum steps
    unsigned maxSteps = 0;
    // and the value that took that many steps
    unsigned valueWithMaxSteps = start;

    for (unsigned i = start; i <= end; ++i) {
        unsigned steps = 0;
        unsigned value = i;
        while (value != 1) {
            if (value % 2 == 0) {
                value /= 2;
            }
            else {
                value = value * 3 + 1;
            }
            ++steps;
        }
        // check if this value took more steps than any
        // of the past values in the range
        if (steps > maxSteps) {
            maxSteps = steps;
            // keep in mind that "value" will be 1 at this point
            // we need the value of i, which was not affected
            // by the algorithm
            valueWithMaxSteps = i;
        }
    }

    std::cout << "Most steps: " << maxSteps << "\n";
    std::cout << "For value: " << valueWithMaxSteps << "\n";

    return 0;
```

```
}
```

Once again, we will test the program to ensure it works. At this point, all requirements have been met. However, we can clean up the program a bit. First, let's move the greeting message to a separate function. It takes up several lines of the main function, without contributing anything meaningful to anyone reading the code:

```
#include <iostream>

void printGreeting();

int main() {
    unsigned start;
    unsigned end;

    // move the greeting message to a separate function
    printGreeting();

    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    if (start == 0 || end == 0) {
        std::cout << "Invalid usage. 0 is not a valid input value\n";
        return 1;
    }

    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        return 2;
    }

    unsigned maxSteps = 0;
    unsigned valueWithMaxSteps = start;

    for (unsigned i = start; i <= end; ++i) {
        unsigned steps = 0;
        unsigned value = i;
        while (value != 1) {
            if (value % 2 == 0) {
                value /= 2;
            }
            else {
                value = value * 3 + 1;
            }
            ++steps;
        }
```

```
        if (steps > maxSteps) {
            maxSteps = steps;
            // keep in mind that "value" will be 1 at this point
            // we need the value of i, which was not affected
            // by the algorithm
            valueWithMaxSteps = i;
        }
    }

    std::cout << "Most steps: " << maxSteps << "\n";
    std::cout << "For value: " << valueWithMaxSteps << "\n";

    return 0;
}

void printGreeting() {
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";
}
```

Next, let's move the error handling to a separate function. Recall that we can use the `exit()` function from `cstdlib` to exit the program if an error occurs. This will allow us to terminate the program from a non-main function:

```
#include <cstdlib>
#include <iostream>

void printGreeting();
void validateInput(unsigned start, unsigned end);

int main() {
    unsigned start;
    unsigned end;

    printGreeting();

    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    // move validation code to separate function
    validateInput(start, end);

    unsigned maxSteps = 0;
    unsigned valueWithMaxSteps = start;
```

```
    for (unsigned i = start; i <= end; ++i) {
        unsigned steps = 0;
        unsigned value = i;
        while (value != 1) {
            if (value % 2 == 0) {
                value /= 2;
            }
            else {
                value = value * 3 + 1;
            }
            ++steps;
        }
        if (steps > maxSteps) {
            maxSteps = steps;
            // keep in mind that "value" will be 1 at this point
            // we need the value of i, which was not affected
            // by the algorithm
            valueWithMaxSteps = i;
        }
    }

    std::cout << "Most steps: " << maxSteps << "\n";
    std::cout << "For value: " << valueWithMaxSteps << "\n";

    return 0;
}

void printGreeting() {
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";
}

void validateInput(unsigned start, unsigned end) {
    if (start == 0 || end == 0) {
        std::cout << "Invalid usage. 0 is not a valid input value\n";
        // use exit to immediately end the program
        exit(1);
    }

    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        exit(2);
    }
}
```

Now, we can move the logic of performing the Collatz algorithm to a separate function. The only thing that we are interested in from the Collatz algorithm is how many steps it takes to complete, so we will

have it return the number of steps:

```
#include <cstdlib>
#include <iostream>

void printGreeting();
void validateInput(unsigned start, unsigned end);
unsigned countCollatzSteps(unsigned value);

int main() {
    unsigned start;
    unsigned end;

    printGreeting();

    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    validateInput();

    unsigned maxSteps = 0;
    unsigned valueWithMaxSteps = start;

    for (unsigned i = start; i <= end; ++i) {
        // now we can just call the function to
        // count the number of steps
        unsigned steps = countCollatzSteps(i);

        if (steps > maxSteps) {
            maxSteps = steps;
            valueWithMaxSteps = i;
        }
    }

    std::cout << "Most steps: " << maxSteps << "\n";
    std::cout << "For value: " << valueWithMaxSteps << "\n";

    return 0;
}

void printGreeting() {
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";
}

void validateInput(unsigned start, unsigned end) {
    if (start == 0 || end == 0) {
```

```
        std::cout << "Invalid usage. 0 is not a valid input value\n";
        exit(1);
    }

    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        exit(2);
    }
}

unsigned countCollatzSteps(unsigned value) {
    unsigned steps = 0;
    while (value != 1) {
        if (value % 2 == 0) {
            value /= 2;
        }
        else {
            value = value * 3 + 1;
        }
        ++steps;
    }

    return steps;
}
```

We can also move printing the results to a separate function:

```
#include <cstdlib>
#include <iostream>

void printGreeting();
void validateInput();
unsigned countCollatzSteps(unsigned value);
void printResults(unsigned steps, unsigned value);

int main() {
    unsigned start;
    unsigned end;

    printGreeting();

    std::cout << "Enter start of range: ";
    std::cin >> start;
    std::cout << "Enter end of range: ";
    std::cin >> end;

    validateInput();

    unsigned maxSteps = 0;
    unsigned valueWithMaxSteps = start;
```

```
    for (unsigned i = start; i <= end; ++i) {
        unsigned steps = countCollatzSteps(i);

        if (steps > maxSteps) {
            maxSteps = steps;
            valueWithMaxSteps = i;
        }
    }

    // move results to separate function
    printResults(maxSteps, valueWithMaxSteps);

    return 0;
}

void printGreeting() {
    std::cout << "This program will find which positive integer"
        << " in a given range takes the most steps to reach 1"
        << " following the Collatz Conjecture\n";
}

void validateInput(unsigned start, unsigned end) {
    if (start == 0 || end == 0) {
        std::cout << "Invalid usage. 0 is not a valid input value\n";
        exit(1);
    }

    if (start > end) {
        std::cout << "Invalid usage. Start must be less than end\n";
        exit(2);
    }
}

unsigned countCollatzSteps(unsigned value) {
    unsigned steps = 0;
    while (value != 1) {
        if (value % 2 == 0) {
            value /= 2;
        }
        else {
            value = value * 3 + 1;
        }
        ++steps;
    }

    return steps;
}

void printResults(unsigned steps, unsigned value) {
    std::cout << "Most steps: " << steps << "\n";
```

```
    std::cout << "For value: " << value << "\n";
}
```

We've now made `main()` function cleaner, with logic that is easier to follow. Next week, when discussing unit testing, we will see that breaking the program up into smaller functions that handle single tasks make it easier to test programs, as well.

Conclusion

This week, we discussed functions. A function consists of a signature, which specifies the name, return type, and parameters of the function, and a body, which indicates the code that a function actually executes when it is called. C++ provides function declarations and function definitions. The declaration is the function's signature followed by a semicolon, and is used to provide a promise to the compiler that the function definition exists somewhere in the code base. This allows functions to be used before they are defined. The definition consists of the function's signature followed by a block. The block contains the statements that the function executes. When the function is called, the statements in the function's body are executed at that point in the code. The function's parameters are given the values of the arguments that are passed to the function call. Arguments can be variables, literals, function calls, or an expression that combines these.

Predefined functions are provided by the C++ standard library headers that are included with your compiler. Predefined functions can also be included from external libraries, but using external libraries is outside the scope of this course. We will only focus on the standard library functions. We looked at examples from `cmath`, `cstdlib`, and `ctime`.

Refactoring is used to reorganize and clean up code. When writing code, it is best to break down the problem and write code that solves the sub-problems. Once combined, however, the code can become messy and disorganized. Spending time to refactor the code can make it more organized and easier to modify in the future.